

TA-MAS Fellowship Team

Final Roles, Responsibilities & Agent Communication Guide

A consolidated reference summarizing the team's finalized coordination structure and the recommended communication architecture for the multi-agent recruitment system, as agreed upon by the fellowship mentor and the three-student team.

Project	TA-MAS — Talent Acquisition Multi-Agent System
Team Size	3 Fellowship Students
Supervisor	Pawan Sapkota Sharma
Document Date	August 4, 2026

Part 1 — Final Roles & Responsibilities

After weighing the tradeoffs between rotating and permanent coordination duties, the team decided that — for a three-person student team still building foundational skills — permanently assigning one coordination responsibility per member is simpler and more sustainable than rotating it every sprint. AI ownership, however, stays equally divided so every member gets deep, hands-on experience with LLMs and agents rather than being boxed into a single technology layer.

Each member therefore owns one major AI agent **and** one lightweight coordination role that keeps the project organized.

Member 1 — Resume Intelligence Lead + GitHub Manager

Owns candidate-facing data extraction and the team's Git workflow

AI RESPONSIBILITY

- Resume parsing
- Information extraction
- Resume embeddings
- Candidate profile generation

COORDINATION ROLE

- Manage the GitHub repository
- Review pull requests
- Maintain branch strategy
- Resolve merge conflicts (with author's help)
- Enforce commit message conventions

Member 2 — Recruiter Decision Lead + Jira Manager

Owns candidate ranking logic and sprint/task tracking

AI RESPONSIBILITY

- Job-resume matching
- Candidate ranking
- Recommendation engine
- Evaluation metrics

COORDINATION ROLE

- Create and organize Jira tasks
- Plan sprint boards
- Track progress
- Record meeting minutes
- Monitor deadlines

Member 3 — Interview & Communication Lead + Integration Manager

Owns candidate-facing communication and end-to-end system integration

AI RESPONSIBILITY

- Interview question generation
- Candidate summaries
- Communication agent
- Feedback generation

COORDINATION ROLE

- Integrate all agents
- Maintain FastAPI endpoints
- Manage Docker Compose
- Run end-to-end testing
- Ensure the complete system works together

Shared Responsibilities (All Members)

- Literature review and research paper discussions
- Prompt engineering
- RAG implementation
- Code reviews
- Testing
- Documentation
- Presentation preparation and final report writing

A Note on the Integration Manager Role

To prevent this role from becoming a bottleneck, the Integration Manager is not responsible for fixing every bug that surfaces during integration. Their job is to coordinate integration, maintain the shared interfaces between agents, run integration tests, and identify where problems occur. If an individual agent breaks the integration, that agent's owner fixes it — the Integration Manager simply verifies the whole system works again afterward.

Why This Structure Works

- Each member owns one major AI component, giving everyone real agent-building experience.
- Each member owns one operational responsibility, so the team stays organized without a single bottleneck.
- No one becomes “the backend developer” or “the frontend developer” at the expense of AI/LLM exposure.
- Fixed ownership (rather than weekly rotation) lets each member build deep expertise in their area over the course of the fellowship.

Part 2 — Agent Communication Architecture

Based on the team's submitted TA-MAS architecture diagram (11 agents spanning input understanding, sourcing, parsing, scoring, fairness auditing, ranking, engagement, and ATS integration, coordinated by a lead Orchestration Agent), the core recommendation is to **avoid direct agent-to-agent communication wherever possible**. Letting Agent 1 call Agent 2, which calls Agent 3, and so on, creates tight coupling, makes debugging difficult, and reduces modularity.

Instead, the **Orchestration/Lead Agent** should coordinate the entire workflow, while individual agents exchange information through structured outputs and shared storage rather than messaging each other directly.

Recommended Communication Flow



Every agent follows the same lifecycle: (1) receive a task from the Orchestrator, (2) perform its task, (3) store or return results as structured data, and (4) let the Orchestrator decide which agent runs next.

Communication Pattern Across the Pipeline

Agent	Consumes	Produces
Job Understanding Agent	Job Description	JobRequirements
Candidate Sourcing Agent	JobRequirements	CandidateList
CV Parsing Agent	CandidateList	ParsedCandidate
Candidate Profile Builder	ParsedCandidate	CandidateProfile
Matching Agent	CandidateProfile + JobRequirements	MatchingScore
Fitment Scoring Agent	MatchingScore	FitmentScore

Agent	Consumes	Produces
Fairness & Bias Audit Agent	FitmentScore	FairScore
Ranking & Shortlisting Agent	FairScore	RankedCandidates
Engagement Sequencing Agent	RankedCandidates	EmailDraft / InterviewSchedule
ATS Integration Agent	RankedCandidates + CommunicationLogs	Updated ATS record

Structured Message Contracts

Agents should never exchange free-text, conversational messages. Every hand-off should use a structured JSON object (or an equivalent Pydantic model in Python), so each agent knows exactly what schema to expect from the one before it. Example of a candidate record passed between agents:

```
{
  "candidate_id": 25,
  "skills": ["Python", "LangChain", "SQL"],
  "experience": 3,
  "score": 87.2
}
```

Standard Response Envelope

Every agent, regardless of function, should return results in the same envelope so the Orchestrator can log and route them consistently:

```
{
  "status": "SUCCESS",
  "agent": "ResumeAgent",
  "input": "candidate_42.pdf",
  "output": {...},
  "confidence": 0.94,
  "timestamp": "..."
}
```

Shared Storage Layer

Rather than passing state directly between agents, all important outputs are persisted centrally so any agent can resume work, failed tasks can be retried, and the full workflow stays traceable.

Store	Used For
PostgreSQL	Structured workflow state — candidates, applications, scores, interviews, status
Vector DB (ChromaDB / Qdrant)	Semantic retrieval — resume embeddings, job embeddings, knowledge retrieval
Neo4j (Graph)	Relationships — candidate has_skill, worked_at, matches_job
File Storage	Original PDFs, DOCX files, and images

Mapping to LangGraph

If the team builds the orchestration layer in LangGraph, this architecture maps directly onto a linear graph, with each pipeline stage as an independent node and the shared graph state carrying structured outputs between them:

```
START → Job Understanding → Candidate Sourcing → Resume Parsing → Profile Builder  
→ Matching → Fitment → Fairness → Ranking → Engagement → ATS Integration → END
```

Governing Principles

- Only the Orchestration Agent controls the workflow — individual agents should not directly invoke one another except in carefully justified cases.
- Every agent communicates through versioned JSON schemas (or Pydantic models), making interfaces explicit and easy to validate.
- All important outputs are persisted in shared storage (PostgreSQL, vector database, or file storage) so the workflow is resumable, retryable, and traceable.

This event-driven, orchestrated design keeps agents loosely coupled, easier to test independently, and much simpler to extend. Adding a future agent — for example, a Salary Negotiation Agent or Reference Check Agent — would only require updating the orchestrator and defining the new agent's input/output schema, without touching any existing agent.

This document consolidates the team's finalized decisions on coordination roles and multi-agent communication design as of August 4, 2026. It is intended as a working reference for the TA-MAS fellowship team and their mentor throughout development.